01 / USER GUIDE

DBETL User Guide

A practical guide to connecting databases, mapping columns, verifying SQL, and loading reviewed records.

The workflow
Connect to DBs > Mapping > Generate Query Preview > optionally Edit Query and Verify Query > Loading > Start Data Load > review > Confirm Insert.

What the application does

DBETL is a browser-based tool for moving records between existing Azure SQL / SQL Server tables. It checks directional datatype compatibility, supports simple mapping formulas and an SQL editor, and separates reading records from confirming a transactional insert.

Page	Use it for
Home /	Open the three workflow steps from the landing page.
Connect to DBs /connect	Enter two database connections and load table metadata.
Mapping /mapping	Select tables, map columns, resolve compatibility issues, and generate or edit SQL.
Loading /loading	Fetch query results, review records, and confirm insertion.

Find the instructions you need

Guide section	Page
Install and run locally	2-3
Connect source and target databases	4
Map columns and resolve validation results	5
Use transformation formulas	6
Edit SQL, use suggestions, and verify changes	7-8
Load and review records	9
Complete a sample transfer	10
Troubleshoot and understand current limits	11

Application version: 0.1.0. Prepared 15 October 2026 from the current project source. Examples use fictional table names and credentials.

02 / USER GUIDE

Run the project locally

Start here if the application is not already running on your computer.

Before you start

- Install Node.js and npm. The installed Next.js package requires **Node.js 20.9.0 or later**.

- Have the project folder, including **package.json** and **package-lock.json**. No repository URL is assumed by this guide.

- Use a terminal and a modern web browser. Internet access is needed to install packages; the first build may also download the Google fonts configured by the application.

- To use database features, have reachable SQL databases, SQL usernames/passwords, and suitable table permissions. The UI can start before databases are connected.

1. Open the project folder

Open PowerShell or your terminal in the folder containing package.json. Replace the example path below with your checkout location.

```
cd "C:\path\to\dbetl_"
node --version
npm --version
```

2. Install the project dependencies

Use the lockfile for a reproducible install. If package.json and the lockfile do not match, reconcile them before using npm ci; npm install can be used when intentionally updating the lockfile.

```
npm ci
```

3. Start the development server

Keep this terminal open while using the application.

```
npm run dev
```

4. Open the application

Open http://localhost:3000. If the terminal reports another port, use the URL it prints. Select **Connect to DBs** to begin.

> **Database configuration**
> The current application takes database credentials through its UI. No .env file or database connection environment variable is required for this workflow.

03 / USER GUIDE

Local commands and checks

Use development mode while editing code, or build and run the application in production mode.

Command reference

Command	Purpose
npm ci	Install the dependency versions recorded in package-lock.json.
npm run dev	Run the Next.js development server.
npm run dev -- --port 3001	Use port 3001 explicitly when 3000 is occupied.
npm run build	Compile the application and check TypeScript as part of the production build.
npm start	Serve the production build after npm run build succeeds.
node --test	Run the repository's automated tests from the project root.

Production mode on your computer

Stop the development server with **Ctrl+C** before starting a different server on the same port. Build first, then start:

```
npm run build
npm start
```

Rebuild after source changes before running the production server again. Unlike development mode, a production server does not provide live code updates.

Stop or restart

Press **Ctrl+C** in the server terminal to stop it. Restart with npm run dev, or npm start if a current production build already exists. A server restart invalidates in-memory verified-query registrations and fetched batches.

Useful local troubleshooting

- If node or npm is not recognized, install Node.js and reopen the terminal so PATH changes take effect.

- If localhost does not open, confirm the terminal is still running and use its exact port. Resolve any startup error shown there.

- If a build cannot download fonts, check access to the font service used by next/font/google in src/app/layout.tsx.

- The project defines npm run lint, but currently has no ESLint configuration file. That command needs configuration before it can run successfully.

The application uses Next.js App Router, React, TypeScript, Tailwind CSS, CodeMirror, and the mssql driver. Python and a local SQL Server installation are not needed to run the web UI when using remote Azure SQL databases.

04 / USER GUIDE

Connect two databases

Use Connect to DBs, or the same connection panels shown on the Mapping page.

1. Identify the two roles

Database 1 - Source provides the records to read. **Database 2 - Target** receives records after confirmation. Each connection can point to a different database.

2. Choose a connection method

Use **Connection string** to paste the complete SQL connection string, or **Individual fields** to enter Server, Database, Username, and Password separately. All four connection values are required.

Example connection string

```
Server=tcp:YOUR_SERVER.database.windows.net,1433;
Initial Catalog=YOUR_DATABASE;User ID=YOUR_USERNAME;
Password=YOUR_PASSWORD;Encrypt=True;
TrustServerCertificate=False;
```

Replace every YOUR_... value, join the displayed lines into one connection string, and paste it into the relevant panel. The example contains no working credentials.

3. Load table metadata

Click **Load Tables DB1**, then **Load Tables DB2**. While connecting, the panels show Connecting... / Fetching tables.... Successful connections expose the table selectors and a table count.

4. Select the tables

Choose the source and target from their table selectors. Names include a schema, for example dbo.CustomerSource. Select existing user tables; the application does not create database tables.

Database access needed

- The source login must be able to read source rows and access the table metadata used for validation.

- The target login must be able to access metadata and INSERT into the target. Returning inserted records can also require appropriate SELECT permissions on output columns.

- Allow the local computer's network path through the database firewall. For typical Azure SQL endpoints, check DNS, TCP connectivity, and the configured server port.

> **Connections and navigation**
> Connections remain available while navigating inside the app. They are held in browser memory, so a page reload or a new tab resets them. Changing connection details or selected tables invalidates the current transfer plan; load metadata and prepare the mappings again.

05 / USER GUIDE

Map and validate columns

On Mapping, connect both databases and select their tables before preparing a transfer plan.

1. Choose target columns

For each source row in **Column Mappings**, use **Target for ...** to select the destination. Choose **Not mapped** to omit a source column. A transformation field becomes available when a target is selected.

2. Read the datatype details

Each mapped row shows source and target definitions, such as int, varchar(100), or decimal(12,2), together with nullability and validation messages. Compatibility is directional: the target must be able to preserve the source domain.

Result	Meaning and action
ACCEPT	The source values fit the target under the configured type rules. Review the notes.
WARN	Review the conversion or formula warning. For mapping warnings, select the acknowledgement checkbox before generating SQL.
REJECT	The mapping can lose data or fail. Change the target, formula, or database design.
UNKNOWN	Compatibility is not established, for example because metadata is missing or a type is unsupported. Resolve it before loading.

Rules you will encounter

- Each target can receive only one source column. Duplicate target selections must be resolved.

- Map required target columns that are NOT NULL and have no default. Leave identity, computed, and SQL Server rowversion/timestamp columns unmapped.

- A target default applies to an omitted column. It does not replace NULL explicitly supplied by a mapped nullable source.

- Examples: int to bigint is safe widening; bigint to int can overflow. varchar(100) to varchar(50) is too small. Decimal precision, scale, encoding, collation, and date/time precision also matter.

3. Resolve the summary

Use the highlighted issue links to jump to problem rows. The mapped-column count and issue total appear below the mappings. **Generate Query Preview** is unavailable while unresolved blocking issues remain.

Changing a mapping or transformation clears the previous query/transfer plan and warning acknowledgements for the changed mapping. Generate and, if editing SQL, verify again.

06 / USER GUIDE

Apply transformations

Mapping formulas use the word value to refer to the current source column. They are a small supported formula language, rather than arbitrary SQL.

Source type	Supported input	Effect
Any supported type	Empty field or value	Copy the source value unchanged.
Text	value.Trim()	Remove leading/trailing spaces using SQL TRIM.
Text	value.ToUpper()	Convert text to upper case.
Text	value.ToLower()	Convert text to lower case.
Numeric	value*2 or value+10	Apply one multiplication or addition.
Numeric	value-5 or value/2	Apply one subtraction or division.
Numeric	value%3	Calculate a remainder for supported numeric types.
Numeric	value-value	Use the same source value as the second operand.
Date/time or Boolean	Empty field or value	Copy unchanged; mapping date functions and other formulas are unsupported.

How to use a formula

1. Select a compatible target

Choose a target first, then type the formula in **Transformation for ...**. Follow the type-specific help below the field. Text method names are case-sensitive.

2. Review the calculated risk

Numeric arithmetic can overflow before conversion to the target. Integer division can truncate fractional digits. A divisor based on value can be zero for some rows. Review any warnings and the actual preview values.

3. Generate and inspect

After resolving validation issues and acknowledging mapping warnings, click **Generate Query Preview** to see the corresponding SELECT and INSERT...SELECT preview.

Examples that need correction
value/0 is rejected. value*2+1 is rejected because multiple mapping operations are unsupported. A fractional numeric operand for an integer target can also be rejected. For a more complex supported SQL expression, use Edit Query, verify it, and review the resulting records before confirming insertion.

07 / USER GUIDE

Edit SQL and use suggestions

The generated preview starts read-only. Editing it is optional when the column mappings already express the transfer you need.

1. Generate the preview

Click **Generate Query Preview**. The editor displays a SELECT and an INSERT...SELECT preview. Generating SQL does not fetch or insert data.

2. Enable editing

Click **Edit Query**. The editor supports SQL highlighting, line numbers, undo/redo, bracket matching, and completion suggestions.

Editor action	How to use it
Request suggestions	Type a keyword/name, or press Ctrl+Space.
Choose a suggestion	Use Up/Down, then Enter or Tab to accept. Press Esc to dismiss.
Complete table/column names	Select names from the current source/target metadata. Column suggestions display datatype details; identifiers are inserted in brackets.
Correct a keyword	For a typo such as SELETC, choose SELECT from the correction suggestions. Nothing changes until you accept a suggestion.
Undo / redo	Use the editor's standard platform undo/redo shortcuts, such as Ctrl+Z on Windows.
Cancel edits	Restore the last successfully verified query, or the generated query when no verified query has been saved.

3. Verify the exact query

Click **Verify Query**. Editing is locked during the request. A success or failure popup appears. Read the message and any conversion warnings, then click **Close**.

4. Continue after success

Click **Continue to Data Load**. The verified SELECT is saved with the transfer plan, so its filters and transformations are used to fetch Source Records. Any later edit requires successful verification again.

> **What suggestions do**
> Autocomplete is a typing aid. It does not make an unsupported query valid or replace Verify Query. Do not change values merely to silence an editor suggestion; use the validation result and actual column datatype.

08 / USER GUIDE

Verify SQL and datatypes

Verification compiles SQL for metadata and checks the application's query rules. It does not run the SELECT for rows or execute the INSERT preview.

Supported query shape

- Use one standalone SELECT, or a SELECT followed by a matching INSERT...SELECT. In the full preview, keep both SELECT statements matching and put a semicolon before INSERT.

- Read from the selected source table and target the selected target table. Nested queries against that source are supported; joins and other tables are currently unsupported by preview validation.

- Every returned column needs a distinct name or alias that matches its destination. If a source and target name differ, use an alias such as [SourceName] AS [TargetName]. Include all required target columns.

Example: reject a fractional integer filter

If Id is an int, the comparison in the inner query below is rejected even though SQL Server can otherwise accept such a comparison:

```
SELECT [Id], [Name]
FROM [dbo].[TestData_New]
WHERE [Id] = 2
  AND NOT EXISTS (
    SELECT 1 FROM [dbo].[TestData_New]
    WHERE [Id] = 2.8
  );
```

Replace 2.8 with an appropriate whole-number value, such as 2, then verify again. A decimal value can be valid for a decimal column when it fits the column's precision and scale. The same datatype checks apply to direct comparisons, IN lists, BETWEEN bounds, and nested predicates.

Other checks and limits

- Checks include integer ranges, decimal precision/scale, text literal length, supported date/time and UUID literals, target compatibility, duplicate columns, and obvious zero divisors.

- Quoted numeric strings are rejected for numeric filter columns under these strict rules. Text columns expect quoted text. Use IS NULL / IS NOT NULL instead of = NULL / <> NULL.

- Verification has a 50,000-character query limit and supports a subset of T-SQL. Runtime expressions, actual row values, unique keys, foreign keys, triggers, and other constraints can still fail during loading or insertion.

> **Verification lasts 15 minutes**
> An edited query must be verified again if its registration expires, the server restarts, or the query, relevant mapping, or connection changes. The server checks current source and target metadata again before fetching its results.

09 / USER GUIDE

Load, review, and insert

On Loading, read the selected source-to-target pair before fetching or confirming records.

1. Fetch the records

Click **Start Data Load**. For an edited query, this uses the successfully verified SELECT, including its filters and transformations. An unverified edit blocks fetching. For an untouched generated plan, the application reads source values and applies the configured mappings.

2. Review both tables

Source Records shows the fetched source values for an untouched mapping plan, or the result columns of your verified edited SELECT. **Mapped Records to Insert** shows the target column names and the exact values proposed for insertion. Verified query expressions are not applied a second time.

Review feature	What to expect
Record count	Each table shows the total fetched count. No returned rows means there is nothing to insert.
Pagination	Tables display 25 records per page. Use Previous / Next when more rows are present; scroll to inspect wide tables.
NULL and structured values	NULL is shown explicitly; structured values are displayed as text/JSON where applicable.
Query details	Expand Verified query from Mapping when present, and Executed source SELECT after fetching, to inspect the SQL used.

3. Confirm the write

Click **Confirm Insert** only after reviewing the mapped records. The application rechecks the target schema, inserts the fetched batch inside one transaction, and displays **Inserted Target Records** plus an inserted-row count on success.

4. Handle the outcome

If a row fails, the app attempts to roll back the transaction. If the message says the commit or rollback outcome could not be verified, inspect the target database before starting another batch. Do not assume that retrying is safe.

> **Two separate 15-minute windows**
> Query verification expires 15 minutes after verification. A fetched batch expires 15 minutes after fetching. If a batch expires, fetch it again; if query verification has expired too, return to Mapping and verify first.

The fetched batch is held on the server: Confirm Insert uses the reviewed values rather than rerunning the source SELECT. Repeated confirmation of the same completed batch returns its saved result. A fresh fetch creates a new batch and does not prevent duplicate insertion across batches.

10 / USER GUIDE

A complete example

Use separate existing source and target tables for this walkthrough. All names and records below are illustrative.

Assumed table design

Source: dbo.CustomerSource	Target: dbo.CustomerTarget
Id int NOT NULL	Id int NOT NULL, not identity
Name varchar(100) NULL	Name varchar(100) NULL

Assume source rows include Id 1 / Name Alice and Id 2 / Name Bob. The target must allow inserting Id 2 without a duplicate-key or other constraint conflict.

1. Connect and select

Load both databases' tables. Select dbo.CustomerSource for DB1 and dbo.CustomerTarget for DB2.

2. Map

Map Id to Id and Name to Name. Leave formulas empty. Resolve any compatibility messages, then click Generate Query Preview.

3. Filter the query

Click Edit Query. Replace the preview with this standalone SELECT, keeping aliases that match the target:

```
SELECT [Id] AS [Id], [Name] AS [Name]
FROM [dbo].[CustomerSource]
WHERE [Id] = 2;
```

4. Verify

Click Verify Query and close the success popup. If validation fails, correct the stated problem and verify again. Changing the filter to Id = 2.8 would fail because Id is an integer.

5. Fetch

Continue to Data Load and click Start Data Load. Source Records and Mapped Records to Insert should contain the result of the filter, rather than all source records.

Id	Name
2	Bob

6. Insert only when ready

Review the result and target database, then click Confirm Insert. On success, check the inserted count and Inserted Target Records. Fetching by itself does not write Bob to the target.

> **If your target generates its own key**
> Do not map an identity Id column. Adapt the mapping and SELECT to return only the writable target columns, such as Name. Keep any other required target columns covered. Changing an SQL alias alone does not make a generated column writable.

11 / USER GUIDE

Troubleshooting and limits

Use the on-screen message to choose the next action. Return to Mapping whenever the query or transfer plan needs correction.

What you see	What to do
Tables will not load	Check server/database, SQL username/password, network/firewall access, and metadata permissions. Make sure all required connection-string fields are present.
Generate Query Preview is disabled	Resolve REJECT / UNKNOWN results, map required target columns, and acknowledge mapping warnings. Select at least one mapping.
Query validation failed	Check syntax and aliases, keep both SELECTs matching if using a full batch, and check literals against column datatypes. Use only supported query shapes.
Query unverified, changed, or expired	Return to Mapping, verify the current text, close the popup, then fetch again. A server restart also requires fresh verification.
No records returned	Confirm the source table and WHERE conditions. A valid query can return zero rows; Confirm Insert stays unavailable.
Batch expired	Click Start Data Load again. Re-verify the query first if its verification has also expired.
Insert failed / constraint conflict	Check duplicate keys, target lengths/ranges, NULLs, foreign keys, checks, permissions, and generated columns. Fix the cause before fetching a fresh batch.
Outcome could not be verified	Inspect the target database before retrying. A lost connection during commit or rollback can leave an uncertain outcome.

Current application boundaries

- This is an insert workflow. It does not automatically update existing target rows, merge/upsert, deduplicate new batches, create tables, or schedule transfers.

- There is no load-abort button, progress percentage, or record export in the current UI. Keep the page open while insertion is running.

- Pagination changes the displayed page, not the fetch size: the matching result set is fetched and held in memory. Use selective filters or TOP for manageable batches.

- Full browser reloads reset connections and the transfer plan. Server restarts clear pending batches and verified-query registrations. These are session workflows, not durable saved jobs.

Guide basis: package.json; the installed Next.js package and installation documentation; the current connection, mapping, query-editor, query-validation, data-loading, records-table, and transfer-service source files. This guide documents implemented behavior and does not include real connection details.